Automation strategy for Amazon's checkout

Test Case ID	Test Description	Automate? (Yes/No)	Unit Level	Justification
TC-01	Verify successful login with valid credentials	Yes	E2E	This is our main happy path login flow. Since we test this on every deployment, automating it saves tons of manual regression time.
TC-02	Verify search bar returns relevant product results for valid keywords	Yes	API	Validates the core search functionality by ensuring the search service returns the correct product data payload.
TC-03	Verify serviceability check accepts a valid delivery pincode and provides delivery estimates.	Yes	API	Checks if our pincode backend API returns correct delivery timelines. Better to automate so we don't have to manually type pincodes.
TC-04	Verify the shopping cart badge count increments correctly when "Add to Cart" is clicked.	Yes	Unit	Simple front-end state check. We can catch visual or count bugs quickly at the component unit level.

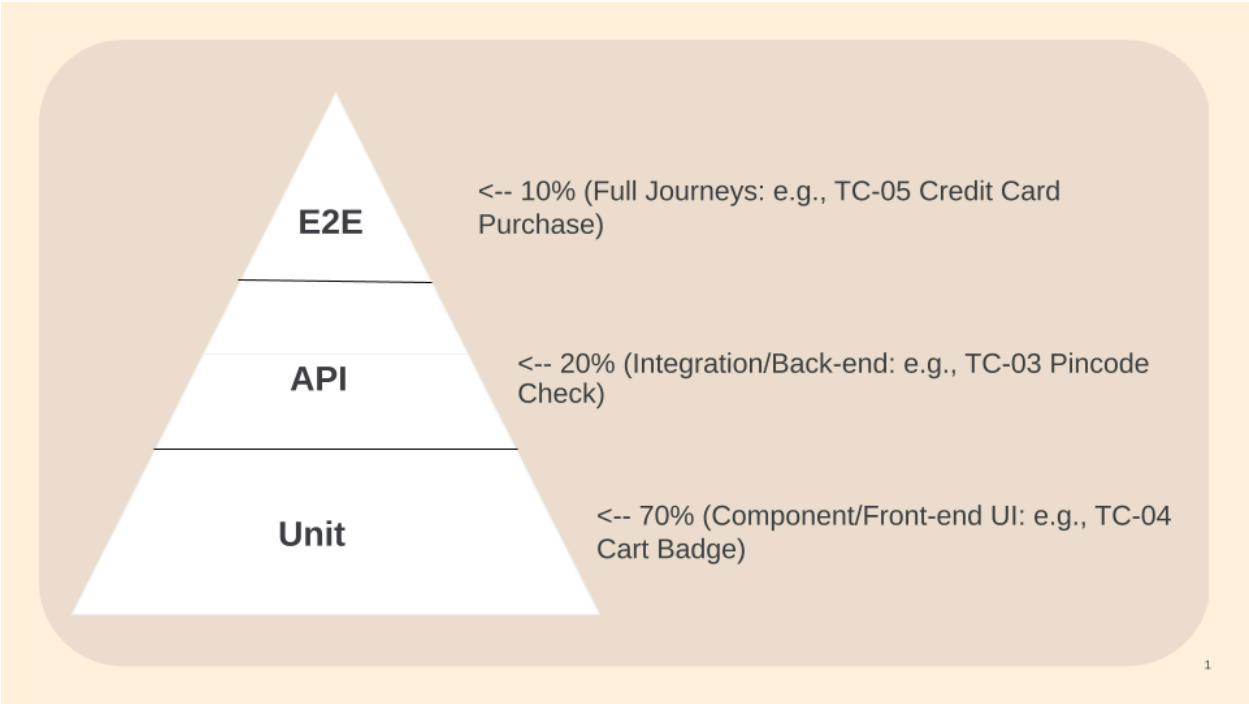
TC-05	Verify that a user can successfully complete a purchase using a valid credit card.	Yes	E2E	This is the most critical smoke test scenario. If checkout breaks, the app loses money, so it must be automated as a full end-to-end flow.
TC-06	Verify that an unauthenticated (logged-out) user is prompted to sign in or transition gracefully when attempting to add an item to the cart.	Yes	E2E	Tests the full user redirection and session handling flow across screens to prevent unauthorized users from bypassing the login wall.
TC-07	Verify that login is denied and an appropriate error message is displayed when using unregistered or incorrect credentials.	Yes	API	Classic negative test case. Essential to ensure the auth API properly blocks bad inputs and returns the correct error codes.
Box-08	Verify that the search results page displays a user-friendly "No results found" message and handled gracefully when a non-existent keyword is entered.	Yes	Unit	Edge case test for empty states. Makes sure the UI component doesn't crash or freeze up when a user types gibberish.

TC-09	This test validates the billing and checkout system's boundary logic when an order's final total drops to exactly $0.00. It ensures that the system bypasses mandatory payment method checks and successfully processes the order instead of throwing a validation error or blocking the user.	Yes	E2E	Tricky boundary value edge case for promo codes/discounts. Automating it ensures the checkout logic handles a free cart all the way to completion.
TC-10	This test validates the system's boundary handling when a user manually inputs a massive quantity (99999) in the cart edit box. It ensures the system gracefully handles the input by resetting it to the seller's maximum limit or available stock instead of breaking the checkout flow or causing an error.	Yes	API	Input validation testing. Automating this at the API layer stops the system from accepting broken numbers before it hits the database.
TC-11	Verify that entering a valid zip code automatically triggers the system to map or validate the correct city and state.	Yes	API	Happy path testing for the address validation API. Automating this ensures our location-matching service isn't broken.

TC-12	Verify that a user can successfully split the final checkout bill between an Amazon gift card and a saved credit card.	No	E2E	This involves a tricky combination of data inputs and complex backend logic. Best to test manually first before scripting.
TC-13	Verify that selecting "Prime Two-Day Shipping" updates the shipping cost to $0.00 and accurately recalculates the final order total.	Yes	API	Core feature for Prime user experience. It checks that the pricing calculator API updates math dynamically when shipping speeds change.
TC-14	Verify that successfully placing an order generates a valid order confirmation number and decrements the warehouse inventory count.	Yes	E2E	Critical data-integrity check. We need to automate this to guarantee that a successful buy updates our backend inventory.
TC-15	Verify that entering an invalid/expired credit card displays a clear, helpful error message and prevents the user from checking out.	Yes	API	Essential negative test case. Automating this protects the checkout wall and ensures error handling doesn't regress.

TC-16	Verify the entire checkout journey smoothly handles a guest user from selecting a new address all the way to the "Thank You" confirmation page.	Yes	E2E	Classic smoke test path. Walking through the whole user flow via automation catches major blockers across different screens.
TC-17	Verify that the Stage-Checkout environment is fully stable, test accounts are loaded with Prime/Guest statuses, and a basic sanity check passes before official execution begins.	No	E2E	This is a human gatekeeper check. We have to manually verify our setup and test data readiness before greenlighting the testing cycle.
TC-18	Verify that when the live payment gateway is simulated as "Down", the backup sandbox/mock system kicks in seamlessly to return a successful transaction response.	No	API	This tests our mitigation strategy for Risk #1. While it deals with a mock backend, executing this verification during an active gateway failure requires manual/strategic setup.

Testing Pyramid



📊 Break Down For Your Report:

- Top Layer: UI / End-to-End (10%)
    - *What it tests:* Complete customer journeys from start to finish.
    - *Example:* TC-16 (Guest checkout flow) or TC-05 (Successful credit card purchase).
    - *QA Summary:* These are high-value smoke tests. They catch major blockers across different screens but run a bit slower.
- Middle Layer: API / Integration (20%)
    - *What it tests:* How backend services and database logic talk to each other.
    - *Example:* TC-03 (Pincode serviceability API) or TC-11 (Zip code mapping service).
    - *QA Summary:* This layer ensures data passes smoothly between endpoints without needing a slow browser to load.
- Base Layer: Unit / Component (70%)
    - *What it tests:* Isolated components, input validations, and small frontend visual states.
    - *Example:* TC-04 (Cart badge incrementing) or TC-08 (Empty search state layout).
    - *QA Summary:* This is the foundation. These tests are incredibly fast, lightweight, and find front-end code regressions instantly.

Tests That Should NOT Be Automated

1. **Visual UI/UX Layout Audits:** Checking if the "Place Order" button color aligns perfectly with the branding guidelines or ensuring text doesn't look awkwardly spaced on unique mobile screens. A human eye catches this instantly, while scripts easily suffer from false positives.
2. **Complex, Rare Multi-Payment Splits (TC-12):** Manually splitting a final bill between an Amazon Gift Card, promotional points, and a saved credit card. The setup overhead, data management, and test case maintenance for this edge case outweigh the automation value.
3. **Ad-Hoc Exploratory Safety Checks:** Navigating random broken links or testing unexpected browser back-button clicking mid-checkout. This relies entirely on human intuition rather than fixed, predictable steps.

Estimate Return on Investment (ROI)

Formula for calculating ROI
ROI = (Hours Saved per Regression Cycle x Cycles per Year) - Initial Setup Hours

Efficiency Breakdown:

- **Manual Regression Time:** It takes a manual QA tester approximately **4 hours** to completely execute, verify, and document all 18 checkout scenarios across multiple targeted test browsers and mobile viewports.
- **Automated Execution Time:** Our automated Selenium/Postman scripts run the identical regression suite in **10 minutes (0.16 hours)** completely unattended.
- **Time Saved Per Cycle:** Each automated execution saves **3.84 engineering hours** (4 - 0.16) by eliminating repetitive manual workflows.
- **Annual Velocity Savings:** Assuming an Agile sprint velocity of 2 deployments per week (104 cycles per year), the automated suite saves **399.36 hours annually** (3.84 x 104).
- **The Investment:** Designing, scripting, debugging, and integrating these 16 core test cases into the CI/CD pipeline requires an initial automation setup effort of **40 hours**.
- **Net First-Year Value:** Subtracting the initial framework setup investment, automating this strategy yields a net savings of **359.36 engineering hours** in the first year alone, proving that the automation suite pays for itself within the first few weeks of implementation.